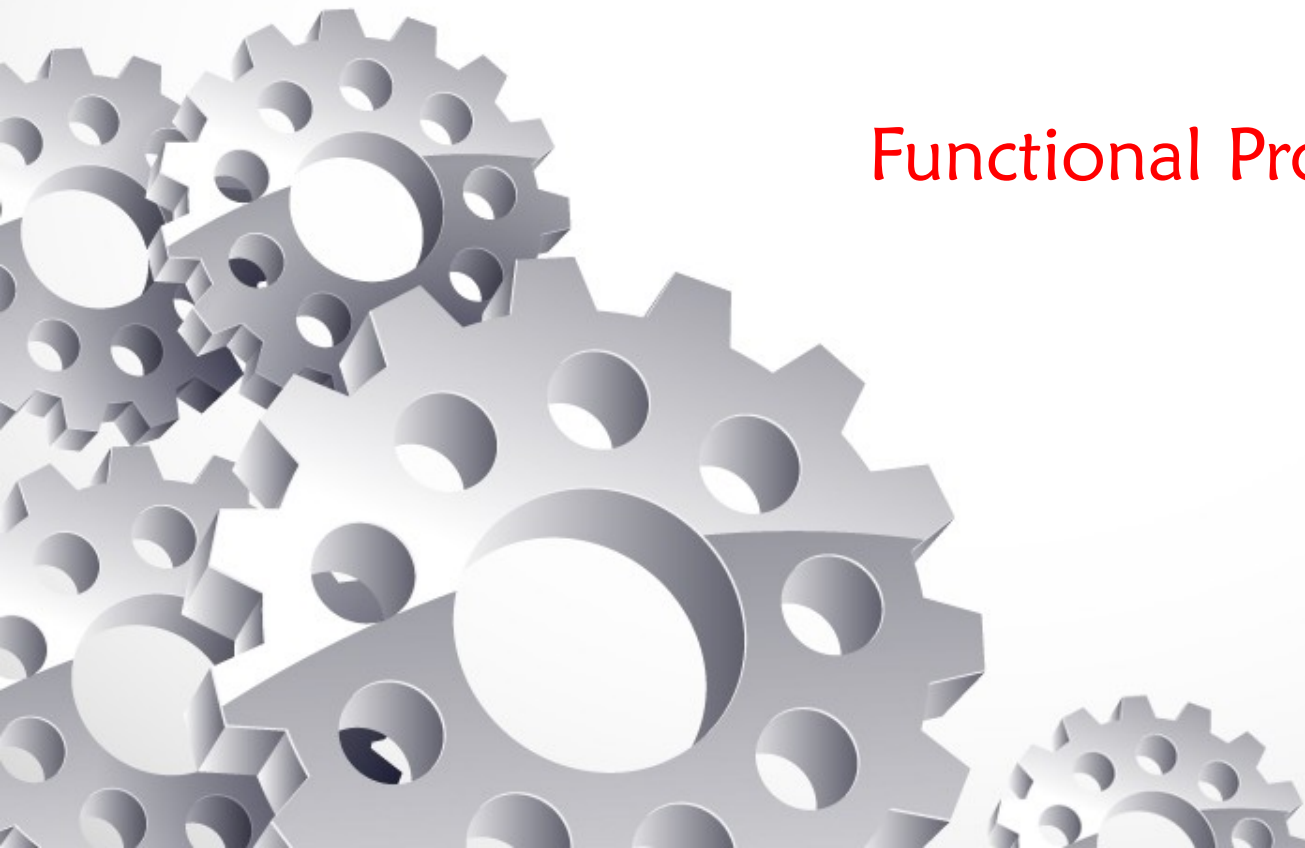


Chapter One

Functional Programming



Java Packages

★ **Package:** A collection of related classes.

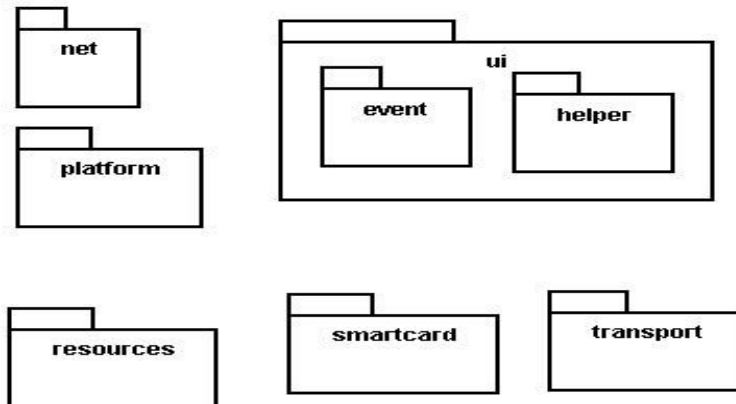
- Can also "contain" sub-packages.
- Packages are Java's way of doing large-scale design and organization.
- They are used both to categorize and group classes.

Uses of Java packages:

- Group related classes together as a *namespace* to avoid name collisions

• Provide a

• Keep piece
size



ion

a manageable

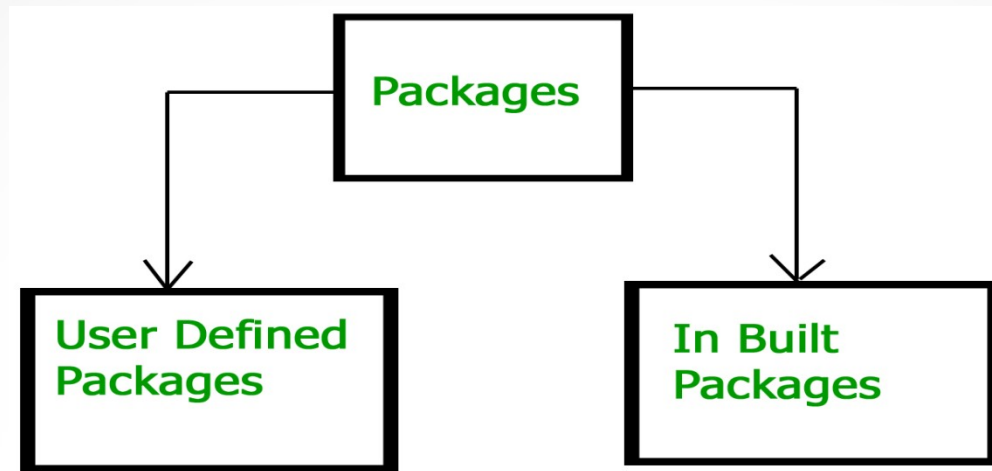
Java Packages

Uses of Java packages:

- ★ Preventing naming conflicts.
 - For example there can be two classes with name Employee in two packages, college.staff.se.Employee and college.staff.ee.Employee
 - import java.util.Date;
 - import my.package.Date;
- ★ Providing controlled access: protected and default have package level access control.
 - A protected member is accessible by classes in the same package and its subclasses.
 - A default member (without any access specifier) is accessible by classes in the same package only.
- ★ Packages can be considered as data encapsulation (or data-hiding).
- ★ Packages naming convention is in reverse order of domain names, i.e., et.edu.se.ceme.aastu



Package Types



Java System Packages & Their Classes

★ Java.lang

- Language support classes
- These are classes that java compiler itself uses & therefore they are automatically imported
- They include classes for primitive types, strings, maths function, threads & exception

Package Types...

Java System Packages & Their Classes

★ Java.util

- Language Utility classes such as vector, hash tables, random numbers, date etc

★ Java.io

- Input/Output support classes
- They provide facilities for the input & output of data

★ Java.awt

- Set of classes for implementing graphical user interface
- They include classes for windows, buttons, list, menus & so on

★ Java.net

- Classes for networking
- They include classes for communicating with local computers as well as with internet servers

★ Java.applet

- Classes for creating and implementing applets

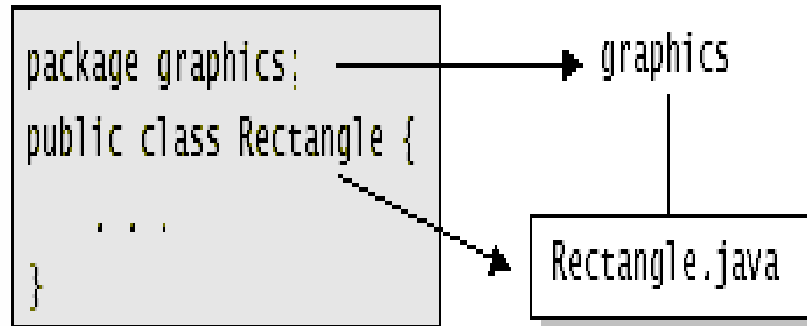


Packages and Directories

★ Package $\leftrightarrow$ directory (folder)

★ Class $\leftrightarrow$ file

- A class named D in package a.b.c should reside in this file:
a/b/c/D.class



- (relative to the root of your project)
- ★ The "root" directory of the package hierarchy is determined by your *class path* or the directory from which *java* was run.

Classpath

★ **Class path:** The location(s) in which Java looks for class files.

★ **It Can include:**

- the current "working directory" from which you ran javac /java
- other folders
- JAR archives
- URLs
- ...

★ **Can set class path manually when running java at command line:**

- `java -cp /home/stepp/libs:/foo/bar/jbl MyClass`

A package declaration

package name;

public class name { ...

Example:

package person.model;

```
public class Student extends Person  
{  
    ...  
}
```

File Student.java should go in folder
person/model/.



Static Import

`import static packageName.className.*;`

- ★ Allows members (fields and methods) defined in a class as public static to be used in Java code without specifying the class in which the field is defined.

Example:

`import static java.lang.Math.*;`

`double angle = sin(PI / 2) + ln(E * E);`

- ★ Static import allows you to refer to the **members of another class** without writing that class's name.
- ★ Should be used rarely and only with classes whose contents are entirely static "utility" code.

Referring to packages

`packageName.className`



Example:

```
java.util.Scanner console = new  
java.util.Scanner(java.lang.System.in);
```

- ★ You can use a type from any package without importing it if you write its full name
- ★ Sometimes this is useful to disambiguate similar names.
 - Example: `java.awt.List` and `java.util.List`
 - Or, explicitly import one of the classes.

The default packages

- ★ Compilation units (files) that do not declare a package are put into a default, unnamed, package.
- ★ Classes in the default package:
 - Cannot be imported
 - Cannot be used by classes in other packages
- ★ Many editors discourage the use of the default package.
- ★ Package `java.lang` is implicitly imported in all programs by default
 - `import java.lang.*;`

Packages access

★ Java provides the following access modifiers:

- *public* : Visible to all other classes.
- *private* : Visible only to the current class (and any nested types).
- *protected* : Visible to the current class, any of its subclasses, and any other types within the same package.
- *default (package)*: Visible to the current class and any other types within the same package.

★ To give a member default scope, do not write a modifier:

```
package person.model;  
public class Student{  
    int points;    // visible to person.model.*  
    String name;   // visible to person.model.*  
}
```

Packages Exercise

- ★ Add packages to the “Student-management-system”:
 - Create a package for core "model" data.
 - Create a package for graphical "view" classes.
 - Create a package for business logic “controller” classes
 - Any general utility code can go into a default package or into another named utility (util) package.
 - Add appropriate package and import statements so that the types can use each other properly.

Collections

- ★ A *collection* — sometimes called a container
- ★ Is simply an object that groups multiple elements into a single unit.
- ★ They are used to store, retrieve, manipulate, and communicate aggregate data.
- ★ Typically, they represent data items that form a natural group, such as
 - a poker hand (a collection of cards),
 - a mail folder (a collection of letters), or
 - a telephone directory (a mapping of names to phone numbers).

What Is a Collections Framework?

- ★ A collections framework is a unified architecture for representing and manipulating collections.

Collections...

- ★ All collections frameworks contain the following:
- ★ **Interfaces:** are Abstract Data Types that represent collections.
 - Interfaces allow collections to be manipulated independently of the details of their representation.
 - In object-oriented languages, interfaces generally form a hierarchy.
- ★ **Implementations:** are the concrete implementations of the collection interfaces.
 - In essence, they are reusable data structures.
- ★ **Algorithms:** are the methods that perform useful computations, such as searching and sorting, on objects that implement collection interfaces.
 - They are said to be *polymorphic*: that is, the same method can be used on many different implementations of the appropriate collection interface.
 - In essence, algorithms are reusable functionality.



Collections...

Benefits of the Java Collections Framework

- ★ The Java Collections Framework provides the following benefits:
- ★ **Reduces programming effort:**
 - By providing useful data structures and algorithms, the Collections Framework frees you to concentrate on the important parts of your program rather than on the low-level "plumbing" required to make it work.
 - By facilitating interoperability among unrelated APIs, the Java Collections Framework frees you from writing adapter objects or conversion code to connect APIs.
- ★ **Increases program speed and quality:**
 - This Collections Framework provides high-performance, high-quality implementations of useful data structures and algorithms.
 - The various implementations of each interface are interchangeable, so programs can be easily tuned by switching collection implementations.
 - Because you're freed from the drudgery of writing your own data structures, you'll have more time to devote to improving programs' quality and performance.

Collections...

Benefits of the Java Collections Framework

★ Allows interoperability among unrelated APIs:

- The collection interfaces are the vernacular by which APIs pass collections back and forth.
- If my network administration API furnishes a collection of node names and if your GUI toolkit expects a collection of column headings, our APIs will interoperate seamlessly, even though they were written independently.

★ Reduces effort to learn and to use new APIs:

- Many APIs naturally take collections on input and furnish them as output.
- In the past, each such API had a small sub-API devoted to manipulating its collections.
- There was little consistency among these ad hoc collections sub-APIs, so you had to learn each one from scratch, and it was easy to make mistakes when using them.
- With the advent of standard collection interfaces, the problem went away.



Collections...

Benefits of the Java Collections Framework

★ Reduces effort to design new APIs:

- This is the flip side of the previous advantage.
- Designers and implementers don't have to reinvent the wheel each time they create an API that relies on collections; instead, they can use standard collection interfaces.

★ Fosters software reuse:

- New data structures that conform to the standard collection interfaces are by nature reusable.
- The same goes for new algorithms that operate on objects that implement these interfaces.

Collections...

- ★ It is a group of elements
e.g student (s1, s2, s3) employee(e1, e2, e3)
- ★ Representing a collection of elements as single object
- ★ It is a single entity that represent multiple objects
- ★ **Collection framework:** to represent a group of elements into a single entity we need certain number of classes and interfaces
 - Provide class and interfaces
 - Define different interfaces and classes by which we can represent a group of elements into a single collection
- ★ **Array**
 - `int a[ ] = new int [100];` // we can store group of elements in array but it is homogeneous
 - We can use Object array to store both homogeneous and heterogeneous elements
e.g. `Objects a[ ] = new Objects [5];`



Collections...

Problems in array

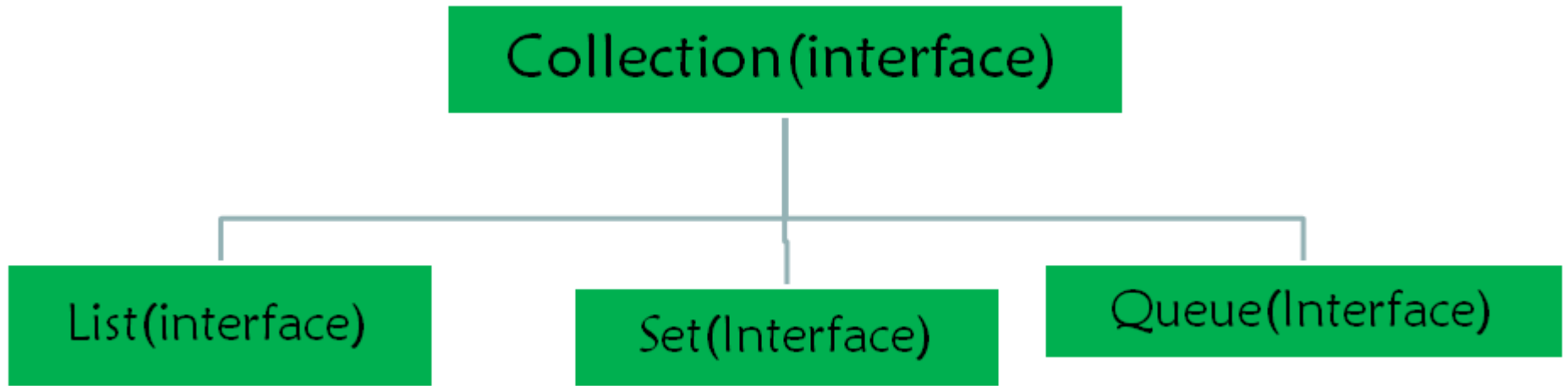
- ★ Fixed in size(not grow able)
 - If extra elements- out of index error
 - If small elements – wastage of memory
- ★ Holds only homogenous data
- ★ Doesn't use built in methods(an underlined methods)
- ★ To a void these problems collection and collection framework
- ★ Collection- is dynamic in size, can hold homogeneous or heterogeneous elements and it can be implemented by using built in (ready made) methods
- ★ Collection Framework; contains a number of classes and interfaces
- ★ Collection(Interface)
 - Root interface for all other related classes and interfaces
 - Contains common methods which can be used for other collection objects



Collections...

★ Collections

- A class from java.util package or it is a predefined class in java.util
 - Contains certain methods that can be used for other collection objects
 - Allow to perform operations in collection object
- E.g for ArrayList (it is a collection object)
- We can use `collections.sort()`, `collections.shuffle()`...



Collections...

List(interface)

- Child interface of collection interface
- Methods available in collection interface will also be available in list interface
- Used for operations where insertion order should be preserved and Duplicate insertion is required
- ★ Classes which implement List interface
 - ArrayList
 - LinkedList
 - Vector(an exchange of stack)

Set(interface)

- ★ Also child interface of collection interface
- ★ Used for operations where insertion order is not preserved and duplicate insertion is allowed
- ★ Classes which implement set interface
 - HashSet, LinkedHashSet
- ★ Collection Framework; contains a number of classes and interfaces
- ★ Collection(Interface)
 - Root interface for all other related classes and interfaces
 - Contains common methods which can be used for other collection objects

Collections...

Queue(interface)

- Child interface of collection interface
- To implement FIFO concepts
- ★ Classes which implement Queue interface
 - Deck

Map(interface)

- Not a child interface of collection interface
- Used for operations where objects are stored in pair of key and value
- Keys can not be duplicated(they are unique)
- Values can be duplicated
- ★ Classes which implement Map interface
 - HashMap
 - LinkedHashMap

N.B: Methods available in collection interface will also be available in child interfaces(List, Set, Queue)



Collections...

The following are methods available in the collection interface

- `add(object o)` – to add new object into the collection
- `addAll(collection c)` – to add all objects of a collection
- `remove(object o)` – to remove an object from a collection
- `removeAll( collection c)` – to remove all objects of a collection
- `retainAll( collection c)` – to remove all objects except the specified group of objects or collection
- `clear()` – clear all objects from the collection
- `isEmpty()` – to check whether a collection is empty or not, it return either true or false
- `size()` – return how many objects are there in the collection
- `contains(object o)` – to check whether a particular object is present or not in a collection
- `containsAll(collection c)` – whether all these group of objects are present in the collection or not

Collections...

The following are methods available in the List interface

- add(index, object o) – insert object at the specified index
 - addAll(index, collection c) – to add all objects of a collection
 - remove(index) – to remove an object in the specified index from the list
- // these methods are adopted from collection interface
- get(index) – to retrieve an object in the specified index
 - set(index, object o) – to rewrite or replace the object found in the specified index by new object

How we can work on ArrayList and LinkedList

syntax of the declaration

ArrayList refernce = new ArrayList();

- Elements are identified by index starting from 0
- Default locations allocated for the arraylist is 10
- Store heterogeneous objects
- ArrayList class can be implemented using a number of methods
e.g. ArrayList al = new ArrayList();

Collections...

How we can work on ArrayList and LinkedList

- To specify the type of Elements we can do as follows

```
ArrayList<string> al = new ArrayList<String>();
```

```
ArrayList<Integer> al = new ArrayList<Integer>();
```



Java Program1

Example:

```
package collectiondemos;
import java.util.ArrayList;
import java.util.List;
public class ArrayListDemo1
{
    public static void main(String[] args)
    {
        // declare ArrayList, we can use one of the following
        // ArrayList al = new ArrayList();
        // ArrayList <Integer> al = new ArrayList<Integer> ();
        // ArrayList <String> al = new ArrayList<String> ();
        // List al = new ArrayList ();
        ArrayList al = new ArrayList();
        //Inset new elements to array List
        al.add(100);
        al.add("welcome");
        al.add(15.5);
    }
}
```



Java Program1....



```
al.add('A');
al.add(true);
System.out.println(al);
//[100, welcome, 15.5, A, true]
// use a method size()
System.out.println("Number of elements in array List:"+al.size());
// remove method
// al.remove(1); // remove element at index 1
al.remove("welcome") // remove element "welcome"
System.out.println("After removing element from arrayList:"+al);
// [100, 15.5, A, true]
// insert new element – add(index, object)
al.add(2, "python");
System.out.println("After insertion:"+al);
//[100,15.5, python, A, true]
// to retrieve an element
//al.get(2);
System.out.println(al.get(2));
```

Java Program1.....



```
// change or replace element
// al.set(2,"C#");
System.out.println("After replacing elements:"+al);
//[100, 15.5,C#, A, true]
// search an element in the arraylist using contains() method – return T/F
//al.contains("C#");
System.out.println(al.contains("C#")); - return true
System.out.println(al.contains("C++")); - return false
// isEmpty();
System.out.println(al.isEmpty()); - return true
//read elements of arrayList
//we can use one of the following
//1----for loop
for(int i=0; i<al.size(); i++)
{
    // al.get(i);
    System.out.println(al.get(i));
}
```

Java Program1....



//2----for each loop

for(object e:al)// one by one each element can be added to the object

{

System.out.println(e);

}

//3----- iterator()- reading using iterator method

Iterator it = al.iterator();

//read each and every element from ArrayList

while(it.hasNext())

{

System.out.println(it.next);// print an element and move to the next

}

}

}

Java Program2

```
package collectiondemos;  
import java.util.ArrayList;
```

```
public class ArrayListDemo2  
{  
    public static void main(String[] args )  
    {  
        ArrayList al =new ArrayList();  
        al.add("X");  
        al.add("Y");  
        al.add("Z");  
        al.add("A");  
        al.add("B");  
        al.add("C");  
        ArrayList al_dup = new ArrayList();  
        al_dup.addAll(al);  
        system.out.println(al_dup); // [X, Y, Z, A, B, C]
```



Java Program2.....



```
al_dup.removeAll(al_dup);
system.out.println("After removing:"+al_dup);
//After removing [ ]
// sort an arraylist sing a method in collection sclass i.e collections.sort()
collections.sort(al);
system.out.println("After Sorting:"+al); [A, B, C, X, Y, Z)
// to sort in reverse order
collections.sort(al, collections.reversorder());
// shuffling using a method from collections
collections.Shuffle(al);
system.out.println("Elements in the array after shuffling:"+al);
// [A, C, X, B, Y, Z]
}
}
```

Java Program3

// To convert array into array list

```
package collectiondemos;
```

```
import java.util;
```

```
public class ArrayListDemo3
```

```
{
```

```
    public static void main(String [] args)
```

```
    {
```

```
        string arr[ ] = {"Dog", "Cat", "Elephant"};
```

```
        for ( String value:arr)
```

```
        {
```

```
            System.out.println(value);    //Dog
```

```
                                           // Cat
```

```
                                           // Elephant
```

```
        }
```

```
        ArrayList al= new ArrayList(Arrays.asList(arr));
```

```
        Systems.out.println(al); //[Dog, Cat, Elephant]
```

```
    }
```

```
}
```



Why Not?

- Functions may act more like procedures
 - a procedure is another unit of modularity
 - the idea behind a procedure is to accomplish one or more related activities
 - the activities should make up some logical goal of the program but may not necessarily be based on producing a single result
 - procedures may return 0 items or multiple items unlike functions
- In languages like C, there are no procedures, so functions must take on multiple roles
 - mathematical types of functions
 - functions that act as procedures
- Such functions can produce side effects
 - mathematical functions do not produce side effects



Functional Design

- Functions should
 - be concise
 - accomplish only a single task or goal
 - return one item
 - if we need to return multiple values, we can wrap them into a list (or other structure)
 - have no side effects
 - because the assignment operations are done by function calls, the function calls must produce side effects in such circumstances, but in general, YOUR code should not produce side effects
 - use parameter passing for communication
 - rather than global variables
 - exploit recursion when possible
 - this simplifies the body of a function and requires fewer or no local variables



Advantages of Functional Programming



- Easier to design
 - everything is a concise module
- Easier to debug
 - if all functions are concise, they contain little code leading to easier debugging and less need to insert break statements to find where things are going wrong
- Can lead to more efficient programs
 - debatable
- Can lead to more efficient use of the programmer's time
- Leads to aesthetically cleaner code
- Functional languages are often interpreted
 - this allows quick implementation and testing
 - this allows for incremental design and implementation of code

Functional Programming

- ★ Modern java supports two primary programming paradigms ; Imperative and Declarative
- ★ Imperative : procedural , object oriented programming
e.g. Procedural: c, FORTRAN,
OOP C++,C#, java
- ★ Declarative: functional and logic programming
e.g. Functional: Haskell, ML
Logic: prolog
- ★ Modern java is a “Hybrid” that combines object-oriented & functional programming
- ★ Java has supported OOP from the very beginning
- ★ It has support for Abstraction, Class, Polymorphism, Inheritance

Overview of the Imperative Programming

- ★ Object oriented programming is an “Imperative” Paradigm
- ★ Imperative programming focuses on describing how a program operates via statements that change its state: e.g. a program consists of commands for the computer to perform

```
List<String> map(List<String> lines, String omit)
{
    List<String> res= new ArrayList<>();
    for(String line : lines)
        if(!omit.equals(line))
            res.add(line);
    return res;
}
```

This program Imperatively remove a given string from a list of strings

Overview of the Imperative Programming...

- ★ Functional programming is a “declarative” paradigm
- ★ Declarative programming focuses on “what” computations should be performed, instead of on “how” to compute them
- ★ e.g. a program express computational logic without describing control flow or explicit algorithmic steps

```
List<String> zap(List<String> lines, String omit)
{
    return lines
        .stream() // convert list into a stream
        .filter(not(omit::equals)) //remove a line in the
            //stream that matches the “omit” parameter
        .collect(toList()); // Collect all non-matching lines
            //into a list & return it to the caller
}
```

This program declaratively remove a given string from a list of strings



Overview of the Imperative Vs Declarative

- ★ Java's object-oriented programming features make code understandable by encapsulating the moving parts
- ★ It's functional programming features make code understandable by largely eliminating the moving part

Key OOP Concepts supported by Java



- ★ It supports key OO concepts and features like
 - ★ Data and control abstraction
 - ★ Supports well-defined APIs & shields programs/programmers from low level implementation details
 - ★ Inheritance
 - ★ Enables systematic reuse of existing methods and fields defined in super classes
 - ★ Polymorphism
 - ★ Dynamically dispatches methods based on runtime type info
- ★ OOP apps written in java are organized in terms of structural elements e.g., classes, interfaces, and packages
- ★ An object is an instance of a class that performs certain operations & interact with other objects
- ★ An object in java resides in a memory location of a computer, correspond to real-world entites

Key OOP Concepts supported by Java...

- ★ Non Object-oriented programming languages organize apps in terms of functional elements

E.g. actions & logic

- FORTRAN, the C programming language

- ★ OO Java Programs also perform actions & contain logic

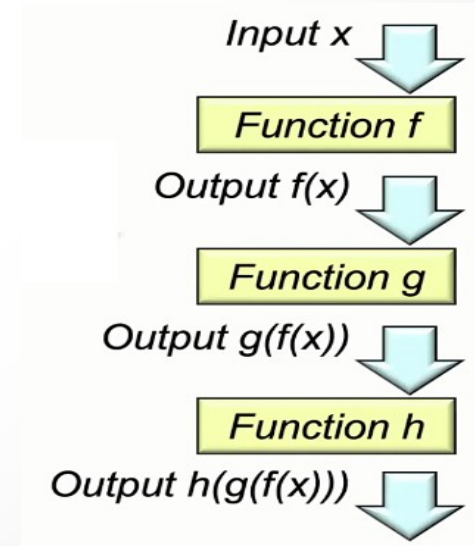
- However, these functional elements don't constitute main focus in the object oriented portions of java

- ★ Java 8 and beyond focus heavily on functional programming



Key Functional Programming Concepts supported by Java

- ★ Functional programming has its roots in lambda calculus,
- ★ Computations are treated as evaluation of mathematical functions
 - We have the input for a function, it does some calculation and give result as an output
 - Ideally, each function is “pure,” i.e., it has no side effects on memory or I/O
- ★ N.B “function composition”, the output of one of function serves as the input to the next function, etc.



Lambda Expression

- ★ In OOP, data will be stored and maintained in the form of class and object
- ★ In Functional programming data will be stored in the form of function and variables
- ★ To improve code optimization/ consize code – we can write small code to achieve large task
- ★ To bring functional programming feature in java (1.8)- Lambda expression
- ★ Lambda expression is anonymous function
 - ★ Anonymous function
 - ★ Doesn't have name
 - ★ Doesn't have access modifier,
 - ★ Doesn't have return type

Lambda Expression...

How to write Lambda expression in java

e.g. the normal method in java

```
1. public void method( )  
    {  
        System.out.println("Welcome");  
    }  
2. public void m2( int a, int b)  
    {  
        System.out.println(a+b);  
    }
```

Lambda expression

```
a. ( ) -> {  
        System.out.println("Welcome");  
    }  
b. (int a, int b ) ->  
    {  
        System.out.println(a+b);  
    }
```

- No need to specify the type of variable or arguments
- Java compiler will automatically find the type of the variable s based on the contest at run time
- So that , (a, b) -> System.out.println(a+b);
- Concise way of writing Lambda expression

Lambda Expression...

How to write Lambda expression in java

e.g. the normal method in java Lambda expression

1. public int m1(int a)	(int a) -> {
{	return(a*a);
return (a+b);	}
}	(a) -> { return (a*a);}; //curly brace is
	// mandatory if there is return statement
	(a) -> a*a; or
	a -> a*a;

- If we have single variable parameter is optional
- Example: validate the following Lambda expression
 1. n-> return n*n; ✗ - b/c curly parenthesis is required
 2. n-> {return n*n;}; ✓
 3. n-> {return n*n}; ✗ - b/c semicolon is missed after n*n
 - n->{n*n;}; ✗ - b/c no need of curly brace
 1. n-> n*n; ✓

Lambda Expression

- ★ There is strong relation ship between functional interface and lambda expression
- ★ Interface- contains only Abstract Method
 - It contains function declaration but not function implementation
- ★ Functional Interface – contains only single Abstract Method
- ★ Before java 1.7 Interface were allowed to contain Abstract Method
- ★ But after java 1.8 in addition to Abstract Method
 - Default methods and
 - Static methods are allowed
- ★ Functional Interface – contains only one AM but we can have any number of default and static methods
- ★ Functional Interfaces in java
 - Runnable → run()
 - Callable → call()
 - Comparable → compareTo()
 - ActionListener → actionPerformed()

Lambda Expression

- ★ To call Lambda expressions we use only functional interfaces
- ★ Without functional Interface we can not involve lambda expressions
- ★ Lambda expressions can be invoked only through Functional Interface

How to write functional interface

interface I

```
{  
    public void method( ); // functional interface  
}
```

interface II

```
{  
    public void method1( );// not functional interface  
    public void method2( );  
}
```

Lambda Expression

interface I

{

public void method1(); // functional interface
default void method2();

{

}

}

Functional Interface Annotation

Example

package demos

@ functional interface // annotation optional

interface Library

{

public void bookLibrary();

}



Lambda Expression

class java implements Library

{

public void bookLibrary()

{

System.out.println(" Java Complete reference is a book");

}

}

public class Test

{

public static void main(String[] args)

{

java Library = new java ();

Library.bookLibrary();

}

}

}



Lambda Expression

package demos

@ Functional interface

interface Library

```
{  
    public void bookLibrary();  
}
```

public class Test

```
{  
    public static void main(String[] args)  
    {  
        Library p= ( )->System.out.println("JCR book is booked....");  
        p.bookLibrary();  
    }  
}
```



Lambda Expression

Example

package demos;

interface Library

{ public void bookLibrary(String source, String destination); }

class Java implements Library

{
public void bookLibrary (String source, String destination)

{
System.out.println("JCR a is book from " + source + " To "
+destination);

}

}

public static void main(String[] args)

{

Library a=new bookLibrary ();

a. bookLibrary("Addiss Ababa", "Dire Dawa");

}



Lambda Expression

Lambda expression equivalent of the above code



```
package demos;
interface Library
{
    public void bookLibrary(String source, String destination);
}

public class Test
{
    public static void main(String[ ] args)
    {
        Library a= (Source, destination)-> System.out.println(" Ola cab ia
        booked from  " + source + "To "+ destination);
        a. bookLibrary("Addiss Ababa", "Dire Dawa");
    }
}
```

Lambda Expression

Without functional Interface we can use Lambda expression

Predefined Functional Interface – using these interface we can write Lambda ex

Java predefined Functional Interfaces

Predicate, Function , Consumer , Supplier

Java.util.function.*; – package contain all these predefined functional interface

Predicate Interface

- Contains a single Abstract Method known as test()
- Prototype of predicate Interface

```
interface predicate(T)
```

```
{  
    public abstact boolean test( ); // takes always one  
    //argument of any type (float, int, String) and it return  
    //boolean value  
}
```

When we do conditional statements, we use predicate functional interface

Lambda Expression

How to use predicate Interface along with test method to invoke lambda expressions

Example

```
interface predicate <T>
{
    public abstract boolean test (T t)
}
```

Package predicates;

Import java.util.function.Predicate;

//predicate ---- > one parameter returns boolean

//use only if you have conditional checks in your program

public class Demo1

```
{
    public static void main(String [ ] args)
    {
        //Ex1
        predicate<Integer> p = i → (i>10);
        System.out.println(p.test(20)); // true
    }
}
```

Lambda Expression

//Ex1

```
predicate<Integer> p = i->(i>10);  
System.out.println(p.test(20)); // true  
System.out.println(p.test(5)); // false
```

//Ex2 : check the length of given string is greater than 4 or not

```
predicate<String> pr=s->(s.length()>4);  
System.out.println(pr.test("Welcome")); // true  
System.out.println(pr.test("abc")); // false
```

//Ex3: print array elements whose size is > from array

```
String name [ ] ={" David ", "Scott", "Smith", "John", "Marry"};  
for(String names:name)  
{  
    if(pr.test(names))  
        {    System.out.println(name);    }  
}  
} // - we can specify multiple conditions in lambda expression
```



Lambda Expression

Example Demo2

```
package predicates;
import java.util.function.Predicate;
class Employee
{
    String ename;
    int Salary;
    int experience;
    Employee(String name, int sal, int exp)
    {
        ename=name;   Salary=sal;   experience= exp
    }
}

public class Demo2
{
    public static void main(String [ ] args)
    { //Example 1
        Employee emp =new Employee("John" , 5000, 5);
```



Lambda Expression

{ //Example 1

```
Employee emp =new Employee("John" , 5000, 5);
```

```
// Emp obj -> return name and salary if sal>30k and exp >3
```

```
Predicate <Employee> pr = e->(e.salary>3000 && e.experience >3);
```

```
System.out.println(pr.test(emp)); //display true
```

```
//Example2, using array List to store a number of employee objects
```

```
ArrayList<Employee> al =new ArrayList <Employee> ( );
```

```
al.add(new Employee ("John", 50000, 5));
```

```
al.add(new Employee ("David", 20000, 2));
```

```
al.add(new Employee ("Scott", 30000, 3));
```

```
al.add(new Employee ("Mery", 40000, 6));
```

```
for(Employee e:al)
```

```
{
```

```
    if(pr.test(e))
```

```
    {
```

```
        System.out.println(e.name + "    " + e.salary);
```

```
    }
```

```
}
```

```
    } } }
```



Lambda Expression

Example Demo3

Join multiple predicate each have one lambda expression

Package predicate;

```
import java.util.function.Predicate;  
    // joining predicate by and, or and negate  
    // p1 ----- checks number is even  
    // p2 ----- checks greater than 50
```

```
public class Demo3
```

```
{  
    public static void main(String [ ] args)  
    {  
        int a[ ]= {5, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65};  
        Predicate<Integer> p1 = i -> i%2==0;  
        Predicate<Integer> p2 = i -> i>50;
```



Lambda Expression

// and

```
System.out.println(" Following are numbers Even & Greater than 50");  
for( int n: a)  
{   if(p1.and(p2).test(n)) // if(p1. test(n) && p2.test(n))  
    {   System.out.println(n); }  
}
```

//or

```
for( int n: a)  
{   if(p1.or(p2).test(n)) // if(p1. test(n) && p2.test(n))  
    { System.out.println(n); // 20 30 40 50 55 60 65 }  
}
```

//negate

```
for( int n: a)  
{   if(p1.negate( ).test(n)) // if(p2.negate( ).test(n))  
    { System.out.println(n);}  
}  
}
```

Lambda Expression

Predefined Functional Interface –Function

Function

- It has only Abstract method known as apply()
- It can take any type of parameter and return a single value of any type

// prototype of this interface

```
interface Function <T, R> // T for the argument, R return value
{
    public R apply(T);
}
```

Used to do some kind of operation

How to work with Functional Interface?

Lambda Expression

Predefined Functional Interface –Function

Example Demo1

```
package functions;
import java.util.function.Function;
public class Demos1
{
    public static void main(String [ ] args)
    {
        Function <Integer, Integer> f=n ->n*n;
        System.out.println(f.apply(5)); // 25
        System.out.println(f.apply(10)); // 100
        System.out.println(f.apply(2)); // 4
//String length
        Function <String, Integer> fn = s->s.length();
        System.out.println(fn.apply("Welvome")); // 7
        System.out.println(fn.apply("Java Programming")); // 16
    }
}
```

Lambda Expression

Object as an argument

Package functions;

Import java.util.function.Fucntion;

class Employee

```
{  
    String ename;  
    int Salary;  
    Employee(String ename, int Salary)  
    {
```

```
        this.ename= ename; // class elements & local variables the same  
        this.Salary= Salary;  
    }
```

Public class Demo2

```
{  
    public static void main(String [ ] args)  
    {    ArrayList <Employee> emplist =new ArrayList<Employee> ();
```

Lambda Expression

Object as an argument

```
emplist.add(new Employee("David", 50000));  
emplist.add(new Employee("John", 30000));  
emplist.add(new Employee("Mary", 20000));
```

```
Function<Employee, Integer> fn= e-> {  
    int sal =e.Salary;  
    if(sal>=10000 && sal <= 20000)  
        return(sal*10/100);  
    else if(sal>20000 && sal <= 30000)  
        return(sal*20/100);  
    else if(sal>30000 && sal <= 50000)  
        return(sal*30/100);  
    else  
        return (sal*40/100);  
}
```

Lambda Expression

Object as an argument

Output

```
for(Employee emp : emplist)
{
    int bonus=fn.apply(emp);
    System.out.println(emp.ename + " " + emp.Salary);
    System.out.println("Bonus is:" + bonus);
}
```

David 50000
Bonus is : 15000
John 30000
Bonus is : 6000
Mary 20000
Bonus is: 2000

Output

// Let us add predicate Interface

```
Predicate<Integer> p =b-> b>5000;
```

```
for(Employee emp: emplist)
```

```
{
    int bonus=fn.apply(emp);
    if(p.test(bonus)
    {
        System.out.println(emp.ename + " " + emp.Salary);
        System.out.println("Bonus is :"+ bonus);
    } } }
```

David 50000
Bonus is : 15000
John 30000
Bonus is : 6000

Lambda Expression

To combine or chain functions there are two methods

Function Chaining ---- `andThen( )`, `compose( )`

Example Demo3

```
package functions;
import java.util.function.Function;
public class Demo3
{
    public static void main(String [ ] args)
    {
        Function <Integer, Integer> f1 = n -> n*2;
        Function <Integer, Integer> f2 =n->n*n*n;
        System.out.println(f1.andThen(f2).apply(2));
    }
}
```

/* first the first lambda expression is executed and the result will be applied on the second Lambda expression and the result will be 4 – for the first LE after 64 form the second LE */

Lambda Expression

To combine or chain functions there are two methods

Function Chaining ----> andThen(), compose()

Example Demo3

```
System.out.println(f1.compose(f2).apply(2));
```

```
// first the second lambda expression is executed and the result will be  
passed as an argument for the first LE
```

```
}
```

```
}
```

Difference between Predicate and Function

Predicate- we have to pass single argument of any type and boolean type
value will be returned

- we have a method test() with predicate

Function – we have to pass parameter type, return type

- any parameter can be passed and any value will be returned
- we have apply () method in function



Lambda Expression

Difference between Predicate and Function

Predicate- used for conditional statement and Function used to process something and get result

Predefined Functional Interface

Consumer - take a single parameter as input but it doesn't return any value
- it has an accept() method

Supplier - it doesn't take any parameter but it return any type of value or object, it has get() method

Example for Consumer LE

```
package consumer;  
import java.util.function.Consumer;  
public class Demo1  
{  
    public static void main(String [ ] args)  
    {  
        Consumer<String> c= s-> System.out.println(s);  
        c.accept("Welcome");  
    } }
```

Lambda Expression

Combine Predicate , Function and consumer LE

- work on Employee object to – calculate Bonus
 - check Bonus > 5000
 - print emp detail

Example Demo2

Package consumer;

class Employee

{

String ename;

int Salary;

String gender;

Employee (String ename, int Salary, String gender)

{

this.ename=ename;

this.Salary=Salary;

this.gender=gender;

}



Lambda Expression

```
public class Demo2
```

```
{  
    public static void main(String [ ] args)  
    {
```

```
        ArrayList<Employee> emplist = new ArrayList<Employee> ( );  
        emplist.add(new Employee("David", 50000, "Male"));  
        emplist.add(new Employee("John", 30000, "Male"));  
        emplist.add(new Employee("Mary", 20000, "Female"));  
        emplist.add(new Employee("Scott", 60000, "Male"));
```

```
// Function -----task1
```

```
    Function<Employee, Integer> f= emp->(emp.salary*10)/100;
```

```
//Predicate ----- task 2
```

```
    Predicate<Integer> p = b-> b>=5000;
```

```
//Consumer ----- task 3
```

```
    Consumer<Employee> c=emp->{  
        System.out.println(emp.ename);  
        System.out.println(emp.Salary);  
        System.out.println(emp.gender);    };
```



Lambda Expression

```
for(Employee e:emplist)
```

```
{
```

```
    int bonus=f.apply(e);
```

```
    if(p.test(bonus))
```

```
    {
```

```
        c.accept(e);
```

```
        System.out.println("Employee bonus:" +bonus);
```

```
    }
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

David

50000

Male

Employee Bonus is : 5000

Scott

Male

Employee Bonus is: 6000



Lambda Expression

Chaining Consumer with example

Package consumer;

import java.util.function.Consumer;

public class Demo3

```
{  
    public static void main(String [ ] args)
```

```
{  
    Consumer<String> c1= s->System.out.println(s+ " is white");  
    Consumer<String> c2= s->System.out.println(s+ " has four legs");  
    Consumer<String> c3= s->System.out.println(s+ " eating grass");  
    c1.andThen(c2).andThen(c3).accept("Cow);
```

/* or

```
Consumer<String> c4=c1.andThen(c2).andThen(c3);  
    c4.accept("Cow"); */
```

/* or

```
c1.accept("Cow"); c2.accept("Cow"); c2.accept("Cow"); */
```

```
}
```

```
}
```

Output

Cow is White

Cow has four legs

Cow is eating grass



Lambda Expression



Supplier – is also a functional interface

- has a method get()
- do not take input parameter but return value of any type
- we will have return type

```
Package Supplier;
```

```
import java.util.function.Supplier;
```

```
Import java.util.Date;
```

```
public class Demo1
```

```
{
```

```
    public static void main(String [ ] args)
```

```
    {
```

```
        Supplier<Date> s = ( )-> new Date( );
```

```
        System.out.println(s.get( ));
```

```
    }
```

```
}
```

Output

Tue Mar 28 11:45 2022